

CS259 Mini-Project 1 — Part 2: Single-Head Attention CUDA Kernels

1. Hardware and Theoretical Bounds

GPU: NVIDIA TITAN V (Volta GV100)

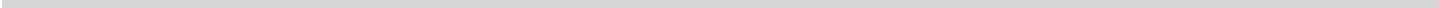
Parameter	Value
FP32 peak compute	13,800 GFLOPS (13.8 TFLOPS)
HBM2 peak bandwidth	652.8 GB/s
L2 cache	4.5 MB
Shared memory / SM	96 KB
SMs	80
Ridge point	21.1 FLOPs/byte

Theoretical FLOPs (D = 64, causal masking):

Workload	Formula	Value
Prefill S=4096	$2 \cdot D \cdot S \cdot (S+1)$	2.15 GFLOP
Prefill S=65536	$2 \cdot D \cdot S \cdot (S+1)$	549.8 GFLOP
Decode C=4096	$4 \cdot C \cdot D$	1.05 MFLOP
Decode C=65536	$4 \cdot C \cdot D$	16.8 MFLOP

Theoretical arithmetic intensity (assuming each matrix element read once):

Workload	I_arith	Region
Prefill S=4096	537 FLOPs/byte	compute-bound
Prefill S=65536	8590 FLOPs/byte	compute-bound
Decode (any C)	0.52 FLOPs/byte	memory-bound



2. Implementation Overview

2.1 Prefill Kernels

prefill_naive (baseline, $S \leq 4096$ only)

- Grid: (S,) — one block per query row. Block: 256 threads.
- Allocates an $S \times S$ global score buffer. For $S=4096$ this is 67 MB; for $S=65536$ it would require 17.2 GB, exceeding TITAN V's 12 GB HBM — this kernel is skipped for the large-S workload.
- Three passes: (1) dot-products $\rightarrow$ score buffer, (2) global max + exp/sum for softmax, (3) weighted V accumulation.

prefill_flash (tile = 16)

- Implements Flash Attention with online softmax, eliminating the score buffer entirely.
- Grid: (S,), Block: D=64 threads — one thread per output dimension.
- Loads K/V in tiles of 16 rows into shared memory. For each key within a tile, the score (a dot product across all D dimensions) is reduced via `__shfl_down_sync` (intra-warp) and a two-entry shared array `swarp[2]` (cross-warp), then the running state (m, sum, out) is updated in registers using the online softmax correction formula: $m_{\text{new}} = \max(m, \text{score})$; $\text{corr} = \exp(m - m_{\text{new}})$; $s = s \cdot \text{corr} + \exp(\text{score} - m_{\text{new}})$.

prefill_flash_v2 (tile = 32)

- Identical algorithm to `prefill_flash` but doubles the tile size to 32 rows.
- The $2\times$ larger tile halves the number of outer-loop iterations and the associated `__syncthreads()` barriers and tile-load overhead, reducing per-query instruction count.
- Shared memory doubles from ~ 8 KB to ~ 16 KB per block, which reduces the number of blocks that can reside simultaneously per SM (occupancy penalty).

2.2 Decode Kernels

decode_naive (baseline)

- Grid: (1,) — single block with 256 threads. Maintains a global score buffer of size C.
- Three-pass softmax over all C context positions. With only one block, only one SM is used, leaving 79 of 80 SMs idle.

decode_flash (2-phase Flash Decoding, slice = 256)

- Phase 1: Each of $\lceil C/256 \rceil$ blocks processes a slice of 256 context positions using online softmax, writing local (m, d, O_partial) to global memory.
- Phase 2: A single block merges all partial results using the same online-softmax merge formula: for each partial block b, $m_{\text{new}} = \max(m, m_b)$, then rescale and accumulate.
- For $C=65536$ this launches 256 blocks, distributing work across all 80 SMs — though at only $\sim 10\%$ occupancy per SM due to the small block size (64 threads vs. the 2048-thread SM capacity).

`decode_flash_v2` (slice = 256, float4 loads)

- Identical 2-phase structure and slice size as `decode_flash` .
- Replaces scalar K/V loads with 128-bit `float4` loads, issuing one memory transaction per 4 floats. This reduces the number of load instructions and issues wider transactions to the memory subsystem.

3. Performance Results

3.1 Benchmark Timing

Kernel	Size	Time (ms)	GFLOPS	Speedup vs naive
prefill_naive	S=4096	7.356	292.0	1.00×
prefill_flash	S=4096	5.444	394.6	1.35×
prefill_flash_v2	S=4096	8.433	254.7	0.87×
prefill_naive	S=65536	—	—	N/A (OOM)
prefill_flash	S=65536	1406.6	390.8	—
prefill_flash_v2	S=65536	2128.0	258.4	0.66× vs flash
decode_naive	C=4096	0.417	2.52	1.00×
decode_flash	C=4096	0.109	9.66	3.83×
decode_flash_v2	C=4096	0.106	9.85	3.91×
decode_naive	C=65536	8.700	1.93	1.00×
decode_flash	C=65536	0.186	90.0	46.8×
decode_flash_v2	C=65536	0.183	91.5	47.5×

3.2 NCU Profiling Data

Kernel	Size	DRAM Read	DRAM Write	ffma count	Measured AI (FLOPs/B)	Occupancy	DRAM BW%
prefill_naive	S=4096	33.88 MB	47.76 MB	1,149M	28.2	47.88%	1.66%
prefill_flash	S=4096	3.15 MB	1.19 MB	5,371M	2,475	29.71%	0.12%

Kernel	Size	DRAM Read	DRAM Write	ffma count	Measured AI (FLOPs/B)	Occupancy	DRAM BW%
prefill_flash_v2	S=4096	3.15 MB	1.20 MB	5,371M	2,469	14.57%	0.08%
prefill_flash	S=65536	742.72 GB	18.21 MB	1,374B	3.70	34.08%	80.90%
prefill_flash_v2	S=65536	11.56 GB	18.21 MB	1,374B	237	15.57%	0.83%
decode_naive	C=4096	2.10 MB	0.51 KB	561K	0.53	7.30%	0.60%
decode_flash	C=4096	2.10 MB	1.28 KB	2,621K	2.49	3.12%	2.64%
decode_flash_v2	C=4096	2.11 MB	1.02 KB	2,621K	2.48	3.12%	2.93%
decode_naive	C=65536	33.96 MB	1.90 MB	8,978K	0.50	7.30%	0.63%
decode_flash	C=65536	33.56 MB	1.44 MB	41,943K	2.40	9.89%	38.29%
decode_flash_v2	C=65536	33.56 MB	1.44 MB	41,943K	2.40	9.96%	40.07%

Measured AI = (ffma_count × 2) / (DRAM_read + DRAM_write)

4. Analysis

4.1 Prefill — necessity of Flash Attention

Standard attention for S=65536 requires storing the full S×S score matrix:

$65536^2 \times 4 \text{ bytes} = \mathbf{17.2 \text{ GB}}$, which exceeds TITAN V's 12 GB HBM. The kernel cannot even be launched. Flash Attention processes K and V in tiles and maintains only a scalar running state (m, d, out) per thread, requiring $O(\text{tile} \cdot D)$ shared memory regardless of S. This makes arbitrarily long sequences feasible.

4.2 Prefill — occupancy from first principles

Occupancy is constrained by shared memory per SM (96 KB) and the smem each block allocates:

Kernel	smem / block	Max blocks / SM (96 KB ÷ smem)	Threads / SM	Theoretical occupancy
prefill_flash	8,208 B	$\lfloor 96 \text{ KB} / 8208 \text{ B} \rfloor = 11$	$11 \times 64 = 704$	$704 / 2048 = \mathbf{34.4\%}$
prefill_flash_v2	16,400 B	$\lfloor 96 \text{ KB} / 16400 \text{ B} \rfloor = 5$	$5 \times 64 = 320$	$320 / 2048 = \mathbf{15.6\%}$

NCU confirms: prefill_flash occupancy = 29.7%, prefill_flash_v2 = 14.6% — closely matching the smem-limited theoretical values. Doubling the tile from 16 to 32 doubles the smem and roughly halves occupancy. This is the root cause of flash_v2's underperformance.

4.3 Prefill — compute-bound vs memory-bound

S=4096 — compute-bound

Both flash kernels show measured AI $\approx 2,470$ FLOPs/byte, far above the ridge point (21.1). DRAM throughput is below 0.12%; essentially all data fits in L1/L2 cache. Performance is compute-limited:

- `prefill_flash` : `sm__throughput` = 57.5% $\rightarrow$ 395 GFLOPS (2.9% of FP32 peak). The bottleneck is the online softmax serial dependency chain: each iteration must complete `m_new` $\rightarrow$ `corr` $\rightarrow$ `es` $\rightarrow$ `s_update` $\rightarrow$ `o_update` before the next key can begin, preventing instruction-level pipelining.
- `prefill_flash_v2` : occupancy halves to 14.6% (from the smem constraint above), `sm__throughput` falls to 36.3% $\rightarrow$ 255 GFLOPS. The fewer barrier syncs from the larger tile cannot compensate for the loss in latency-hiding capacity.

Why flash_v2 has more FFMA than naive (5,371M vs 1,149M)

The flash kernels execute approximately $4.7\times$ more FFMA instructions than naive for the same mathematical output. In addition to the QK dot product (identical to naive), each key processed by flash requires the online softmax update: `s = s*corr + es` (1 FFMA) and `o = o*corr + es*sV` (1–2 FFMA), plus the score scaling. This extra arithmetic is what drives the measured AI to 2,475 FLOPs/byte — the DRAM bytes are the same but the numerator is much larger.

S=65536 — memory-bound despite identical algorithm

This is the most instructive finding. `prefill_flash` achieves 391 GFLOPS at S=65536, nearly identical to its S=4096 result. Yet the bottleneck is completely different:

- DRAM throughput = **80.9%** of peak (≈ 528 GB/s measured)
- DRAM read = **742.72 GB** (vs only 3.15 MB at S=4096)

The measured AI drops from a theoretical 8,590 FLOPs/byte to just **3.70 FLOPs/byte** — below the ridge point. Why? Flash attention reads K and V tile-by-tile for each of the S=65536 query rows. Key j is accessed by queries j, j+1, ..., S-1, an average of $S/2 \approx 32,768$ times. Total K+V DRAM traffic $\approx S^2/2 \times 2D \times 4$ bytes ≈ 1.1 TB, far exceeding the 4.5 MB L2 cache; every tile fetch is a cold miss.

Why DRAM BW = 80.9% but roofline efficiency = only 16.2%

On a log-log roofline, the kernel sits at ($AI=3.70$, 391 GFLOPS) while the memory-bound roofline at that AI predicts $652.8 \times 3.70 = 2,415$ GFLOPS. The gap (16.2%) arises from **causal load imbalance**: block 0 processes 1 key while block 65535 processes 65535 keys. At any given time the active SM set contains a mix of near-finished early blocks and work-heavy late blocks. Early blocks complete almost instantly and their SMs sit idle, pulling down average throughput. The 80.9% DRAM figure is dominated by the final wave of large blocks and does not reflect the steady-state utilization across the full kernel lifetime.

S=65536, flash_v2 — cache efficiency vs occupancy tradeoff

`prefill_flash_v2` at $S=65536$ shows a striking contrast:

- DRAM read = only **11.56 GB** (vs 742.72 GB for flash) — 64× less DRAM traffic.
- The 2× larger tile (32 rows) doubles the temporal locality: a K/V tile loaded by block i covers twice as many subsequent blocks before being evicted from L2, dramatically increasing the L2 hit rate.
- Yet performance is **worse** (258 vs 391 GFLOPS). Occupancy drops from 34% to 16% (smem constraint), halving the number of warps available to hide latency. `sm_throughput` falls from 56.8% to 37.0%. Even though DRAM is no longer the bottleneck, the compute pipeline is severely underutilized.

This demonstrates a fundamental tradeoff: larger tiles improve cache reuse but pay an occupancy tax that can exceed the benefit.

4.4 Decode — memory-bound, multi-block parallelism is key

All decode kernels show measured $AI \approx 0.5\text{--}2.5$ FLOPs/byte, far below the ridge point. Decode is fundamentally memory-bound: with only one query vector, arithmetic is negligible compared to the cost of streaming the entire KV cache from HBM.

Why decode_flash has higher AI than decode_naive (2.49 vs 0.53)

Flash decode performs the same online softmax update per key as flash prefill (extra $\approx 3\text{--}4$ FFMA per key), yielding $\sim 4.7\times$ more total FFMA instructions (2,621K vs 561K) while reading the same DRAM bytes. The AI increases from 0.53 to 2.49 FLOPs/byte — a rightward shift on the roofline. The kernel remains memory-bound ($2.49 \ll 21.1$ ridge), but the additional compute makes slightly better use of each byte fetched.

decode_naive: Grid = (1,) — single block, single SM. For $C=65536$, DRAM throughput = 0.63% despite the kernel being memory-bound. A single block with 256 threads can issue only a limited number of outstanding memory requests, leaving 99.4% of HBM bandwidth unused. This is the classic single-threaded memory bottleneck.

decode_flash: Launches $\lceil C/256 \rceil = 256$ blocks for $C=65536$, distributing work across all 80 SMs. DRAM throughput rises to **38.3%** and performance improves **46.8×** over naive. Having many blocks in flight simultaneously allows hundreds of memory transactions to be outstanding at once, far better utilizing HBM.

decode_flash_v2 (+ float4): Same 256 blocks as flash. The `float4` loads increase L1-texture traffic from 33.70 MB to 98.71 MB — wider vector fetches pull in more cache-line data per instruction, increasing L1 pressure but also coalescing more data per transaction. DRAM throughput rises marginally to **40.1%**, yielding a small but consistent improvement: 91.5 vs 90.0 GFLOPS for $C=65536$.

All decode kernels are severely below DRAM peak (40% best case). The root cause is the inherently small problem: $256 \text{ blocks} \times 64 \text{ threads} = 16,384 \text{ active threads}$, versus TITAN V's theoretical capacity of $80 \text{ SMs} \times 2048 \text{ threads} = 163,840 \text{ threads}$. Occupancy is $\sim 10\%$, leaving most of the GPU's memory-request bandwidth idle.

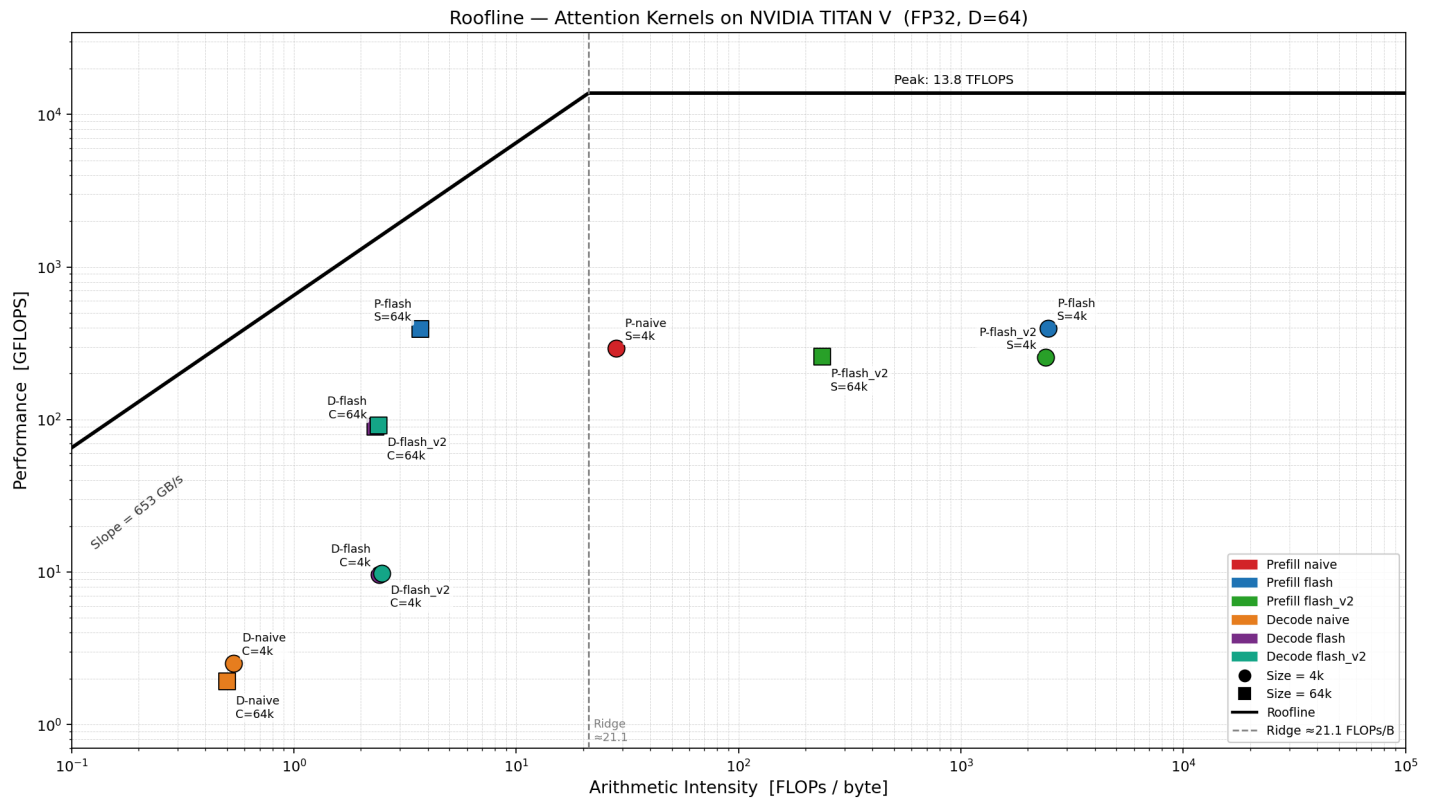
4.5 Why optimizations sometimes fail

Optimization	Expectation	Actual outcome	Root cause
prefill_flash_v2: tile 16→32	Fewer outer iterations → faster	Slower at S=4096 (255 vs 395 GFLOPS)	smem 8→16 KB, occupancy 34%→15%, sm__throughput 57%→36%
prefill_flash_v2 at S=65536	Better L2 cache reuse	Still slower (258 vs 391 GFLOPS) despite 64× less DRAM	Low occupancy outweighs cache gains; sm__throughput 37% vs 57%
decode_flash_v2: float4	More efficient loads, faster	Marginal +1.7% at C=65536	Memory-bound; both kernels read identical DRAM bytes; float4 only marginally helps L1
decode_flash_v2 at C=4096	Measurable speedup	Negligible (9.85 vs 9.66 GFLOPS)	Problem too small; GPU idle time dominates

5. Roofline Summary

Kernel	Measured AI	Bottleneck	% of roofline
prefill_naive S=4096	28.2 FLOPs/B	compute (near ridge)	$292 / 13800 = 2.1\%$
prefill_flash S=4096	2,475 FLOPs/B	compute (deep)	$395 / 13800 = 2.9\%$
prefill_flash_v2 S=4096	2,469 FLOPs/B	compute, low occupancy	$255 / 13800 = 1.8\%$
prefill_flash S=65536	3.70 FLOPs/B	HBM bandwidth (load imbalance limits to 16.2%)	$391 / (652.8 \times 3.70) = 16.2\%$
prefill_flash_v2 S=65536	237 FLOPs/B	compute, low occupancy	$258 / 13800 = 1.9\%$
decode_naive C=65536	0.50 FLOPs/B	HBM, single block	$1.93 / (652.8 \times 0.50) = 0.59\%$

Kernel	Measured AI	Bottleneck	% of roofline
decode_flash C=65536	2.40 FLOPs/B	HBM, multi-block	$90 / (652.8 \times 2.40) = 5.7\%$
decode_flash_v2 C=65536	2.40 FLOPs/B	HBM, multi-block	$91.5 / (652.8 \times 2.40) = 5.8\%$



All kernels remain well below theoretical peak. The primary reasons are:

- Online softmax serial dependency:** $m \rightarrow \text{corr} \rightarrow \text{es} \rightarrow \text{s} \rightarrow \text{o}$ is a 5-step chain with strict data dependencies that prevents instruction-level pipelining, limiting compute throughput even when the kernel is nominally compute-bound.
- Shared-memory-limited occupancy for flash_v2:** 16 KB smem per block allows only 5 blocks/SM vs 11 for flash, halving the warp pool available to hide latency.
- Causal load imbalance at large S:** Block i processes $i+1$ keys; early blocks finish almost instantly while late blocks dominate runtime, preventing sustained peak DRAM utilization.
- Decode problem size:** One query vector vs a large KV cache is too small to saturate 80 SMs; even 256 blocks achieve only $\sim 10\%$ occupancy.

6. Conclusion

Flash Attention is indispensable for large prefill sequences: standard attention requires 17.2 GB for $S=65536$, exceeding hardware limits, while flash attention operates with $O(\text{tile} \cdot D)$ shared memory regardless of S . At $S=4096$, flash attention achieves a **1.35x speedup** over naive by eliminating the score buffer and reducing

DRAM traffic from 81.6 MB to 4.3 MB; the workload is compute-bound at 2.9% of FP32 peak, limited primarily by the online softmax serial dependency chain. At $S=65536$, the bottleneck shifts to HBM bandwidth (80.9% utilization, 528 GB/s measured) because K and V must be re-read for each of the 65,536 query rows, accumulating 742 GB of total DRAM traffic despite each matrix being only 16 MB. Causal load imbalance further limits sustained bandwidth utilization to 16.2% of the memory-bound roofline.

Attempts to improve prefill via a larger tile (flash_v2, tile=32) revealed a fundamental tradeoff: the $2\times$ tile doubles L2 cache reuse (DRAM reads drop $64\times$ at $S=65536$) but halves occupancy due to the doubled shared-memory footprint, resulting in net slowdown. This demonstrates that for kernels with serial dependency chains, occupancy — not cache efficiency — is the primary determinant of throughput.

For decode, multi-block Flash Decoding provides a **47× speedup** by distributing the KV cache across 256 blocks, raising DRAM throughput from 0.6% (single block) to 40% (multi-block). Both prefill and decode remain far from theoretical limits, and future improvements would require breaking the online softmax dependency chain (e.g., via warp-level software pipelining) or migrating to tensor-core-based FP16/BF16 computation.